



Kickoff

Documentação de planejamento da ferramenta que transforma a descrição de um projeto de software em plano de execução.

ESFORÇO ESPERADO

529 h

85% DE CONFIANÇA

553 h

DURAÇÃO

9,2 sem

MARCOS

5

Sumário

01 Visão geral e premissas

Por onde começar, o que assumimos e o que ainda não sabemos

02 Produto

O problema, quem usa, o que está dentro e fora do escopo

03 Roadmap

Marcos, critérios de saída e datas projetadas

04 Backlog

Épicos e tarefas com estimativa de três pontos

05 Riscos

O que pode dar errado e o que fazemos a respeito

06 Decisões técnicas

Arquitetura já definida, com motivo e gatilho de revisão

07 Processo

Cadência, definição de pronto, testes e escopo

Visão geral e premissas

Por onde começar, o que assumimos e o que ainda não sabemos

Documentação de planejamento do próprio Kickoff. A ordem abaixo é a ordem de leitura para quem chega no projeto.

DOCUMENTO	PARA QUÊ	QUEM MANTÉM
produto.md	O problema, quem usa, o que está dentro e fora do escopo, como sabemos que deu certo	Produto
roadmap.md	As fases, os marcos e as datas projetadas	Gerência de projeto
backlog.md	Épicos e tarefas com estimativa de três pontos	Time todo
riscos.md	O que pode dar errado e o que fazemos a respeito	Gerência de projeto
decisoes-tecnicas.md	Decisões de arquitetura já tomadas, com o motivo	Time técnico
processo.md	Como trabalhamos: branches, revisão, pronto, cadência	Time todo

Versão em PDF: [kickoff-planejamento.pdf](#) — os sete documentos num arquivo só, para leitura ou impressão. Para regerar depois de mudar a documentação:

```
python scripts/build-docs-pdf.py
```

O script monta o HTML de impressão; o PDF sai daí com o Chrome em modo headless (veja o cabeçalho do script).

Premissas que sustentam todo o resto

Estas premissas foram assumidas para produzir números. **Se qualquer uma delas for falsa, o roadmap muda** — corrija aqui primeiro e recalcule depois.

- Time de 2 pessoas envolvidoras**, 30h semanais efetivas cada (60h/semana de capacidade). Não estão contadas reuniões longas, suporte e férias.
- Início em 06/08/2026**, com a v0 já pronta e funcionando.
- Ferramenta interna agora, produto depois**. Sem requisito de cobrança nesta fase, mas nenhuma decisão de arquitetura pode inviabilizar a virada.
- 3 pessoas usando no primeiro trimestre**, em regime de teste.

5. **Uma pessoa decide escopo.** Sem isso, as estimativas não valem — a maior fonte de atraso em projeto pequeno é decisão travada, não código difícil.

Decisões tomadas em 05/08/2026

As quatro perguntas em aberto foram respondidas. O que mudou:

PERGUNTA	RESPOSTA	CONSEQUÊNCIA NO PLANO
Interna ou produto?	Interna agora; venda depende do desempenho	Cobrança fica fora da v1. Isolamento por usuário continua obrigatório — é o que torna a virada possível sem reescrita
Orçamento de API por plano?	Sem teto por enquanto	Podemos rodar em esforço alto sem restrição. Continuamos medindo — o número vira crítico no dia da decisão de vender
Integrar com Jira, Linear ou Azure DevOps?	Não. Construir a nossa, no mesmo patamar, menos burocrática e mais intuitiva	Mudança grande: entra o épico E7 (execução do plano) e o não-escopo muda. +106h, roadmap vai de 7,4 para 9,2 semanas
Quantas pessoas no 1º trimestre?	3, em teste	Espaço compartilhado pode ser simples: convite direto, sem hierarquia de organização, papéis ou cotas

A terceira resposta é a de maior impacto e merece ser lida com atenção: o Kickoff deixa de ser só um **gerador de plano** e passa a ser também o lugar onde o plano é **executado**. Isso quase dobra a superfície do produto. A aposta que a justifica está em **produto.md**: planejar e executar no mesmo lugar elimina a tradução manual entre a ferramenta que planeja e a que executa — que é onde os planos costumam morrer.

Perguntas ainda em aberto

- Quando a decisão de vender for tomada, o isolamento passa a ser por organização, não por usuário. Vale antecipar o modelo de dados agora ou aceitar a migração depois? (Recomendação: aceitar depois — antecipar multi-organização sem cliente pagante é custo especulativo.)
- Qual das três pessoas do teste tem autoridade para decidir escopo?

Estado atual: v0 entregue

Funciona hoje, ponta a ponta:

- Formulário de descrição livre → plano gerado → visualização e edição.
- Épicos, tarefas, riscos, marcos, premissas e perguntas em aberto.
- Estimativa de três pontos com agregação PERT e faixa de 85% de confiança.
- Recálculo ao vivo quando se muda estimativa, capacidade ou estado de tarefa.
- Exportação em Markdown.
- Banco embutido (PGLite), sem nada para instalar.

Ainda não validado: o caminho de geração com Claude nunca rodou de verdade — falta a chave de API. Todos os planos gerados até agora vieram do motor heurístico local. Isso é o item número um do backlog, e é bloqueante.

Produto

O problema, quem usa, o que está dentro e fora do escopo

O problema

Planejar um projeto de software é caro e a maioria dos times faz mal. O padrão que se repete:

- **A quebra do trabalho é rasa.** O plano tem "fazer o backend", não as trinta coisas que "fazer o backend" significa. O que não foi escrito não foi estimado.
- **O que sempre é esquecido continua sendo esquecido.** Migração de dados, autorização, observabilidade, deploy, treinamento, documentação. Todo projeto descobre isso na metade.
- **A estimativa é um número único.** "Seis semanas" vira compromisso, sem ninguém dizer se é 6 no melhor caso ou 6 na média.
- **Riscos são genéricos.** "Prazo apertado" não é risco, é reclamação.
- **Premissas ficam na cabeça de quem planejou.** Quando a premissa cai, o plano cai junto e ninguém sabe por quê.

Ferramentas de gestão (Jira, Trello, Linear) executam bem um plano que já existe. Nenhuma delas ajuda a **produzir** o plano.

E há um segundo problema, que só aparece depois: mesmo quem planeja bem acaba **traduzindo o plano à mão** para a ferramenta onde o trabalho realmente acontece. Nessa tradução o plano perde as estimativas de três pontos, perde as premissas, perde os riscos — e vira uma lista de tarefas sem memória de por que foram criadas. Duas semanas depois, ninguém sabe mais qual era o plano.

Para quem

PESSOA	O QUE ELA QUER	COMO O KICKOFF ENTREGA
Dev tech lead	Não descobrir na semana 8 que faltou migração de dados	Quebra do trabalho que inclui as frentes que se esquece por padrão
Gerente de projetos	Um prazo que possa defender e explicar	Faixa de confiança com o cálculo à vista, não número mágico
Pessoa que patrocina o projeto	Saber o que está sendo assumido	Premissas e perguntas em aberto explícitas no documento

Proposta de valor

Você descreve o projeto em texto livre. Em menos de dois minutos tem um plano de execução completo — e continua sendo dono dele, porque tudo é editável.

Quatro coisas nos diferenciam:

1. **A saída é estruturada e viva.** Não é texto: são objetos que você edita, e editar uma estimativa recalcula o prazo do projeto inteiro na hora.
2. **A estimativa é honesta por construção.** Três pontos por tarefa, agregação PERT, faixa de 85%. A ferramenta torna difícil produzir um prazo otimista.
3. **O que não se sabe fica visível.** Premissas e perguntas em aberto são seções de primeira classe, não um parágrafo no fim.
4. **Planejar e executar no mesmo lugar.** O plano não é exportado para outra ferramenta: ele *vira* o quadro de execução, sem perder as estimativas, as premissas e os riscos que o originaram.

A aposta por trás do item 4

Decidido em 05/08/2026: **não vamos integrar com Jira, Linear ou Azure DevOps.** Vamos construir a camada de execução, inspirada nelas, mas menos burocrática.

O argumento a favor: a tradução manual do plano para a ferramenta de execução é onde o plano morre. Se o quadro de execução é o próprio plano, cada tarefa concluída atualiza o prazo automaticamente, e o replanejamento passa a ter dados reais em vez de suposição.

O argumento contra, que precisa ser dito: **Jira e Linear têm anos de trabalho acumulado.** Não vamos empatar com eles em recursos, e não devemos tentar. A única forma de isso dar certo é sendo deliberadamente mais simples — ver a seção de não-escopo abaixo, que é o que impede a gente de virar um Jira pior.

Escopo da v1

Dentro:

- Geração de plano a partir de descrição livre, com Claude, com qualidade comprovada por avaliação (hoje só temos a heurística testada).
- Contas de usuário e isolamento de dados entre pessoas.
- Espaço de trabalho compartilhado: mais de uma pessoa vê o mesmo projeto.
- Edição completa do plano gerado: tarefas, estimativas, estado, capacidade.
- **Execução do plano:** quadro por estado com arrastar e soltar, responsável por tarefa, comentários e a visão "o que é meu agora".
- Exportação em Markdown.
- Rodar em Postgres gerenciado, com migrações versionadas e monitoramento.

Não-escopo da v1

Escrito para ser citado quando alguém pedir. Cada item aqui é uma decisão, não um esquecimento. **Agora que vamos ter execução própria, esta lista é o que nos impede de virar um Jira pior** — trate cada item como uma linha que não se cruza sem discussão explícita:

- **Sprints, story points e cerimônia de método.** O quadro é por estado da tarefa, não por iteração. Sem planning poker, sem velocity, sem burndown.
- **Campos personalizados e fluxos configuráveis.** É exatamente isso que torna o Jira burocrático. Os estados são três — pendente, fazendo, feito — e não são configuráveis.
- **Hierarquia organizacional.** Sem times dentro de times, sem projeto dentro de portfólio, sem permissão por campo.
- **Apontamento de horas.** Estimamos esforço; não controlamos ponto.
- **Automações e regras.** "Quando mover para X, faça Y" é a porta de entrada da complexidade infinita.
- **Sincronização com ferramenta de gestão externa.** Decidido: construímos a nossa. Exportar em Markdown continua; sincronizar dos dois lados, não.
- **Cobrança e planos pagos.** Entra quando a decisão de vender for tomada.
- **App mobile nativo.** A tela funciona em celular; app nativo não.
- **Dependências entre tarefas e caminho crítico.** Reconhecemos que falta — está no roadmap, depois da execução. Ver [roadmap.md](#).
- **Colaboração em tempo real** no estilo edição simultânea.

Como saberemos que deu certo

Métricas para medir a partir do marco M2. Nenhuma delas é "número de planos gerados" — gerar é fácil, o que importa é o plano ser usado.

MÉTRICA	ALVO	POR QUÊ
Planos gerados que continuam sendo editados 7 dias depois	> 50%	Plano que ninguém volta a abrir foi entretenimento
Tarefas editadas ou adicionadas por plano	mediana ≥ 5	Mede se a pessoa se apropriou do plano
Tempo da descrição ao plano aprovado	< 30 min	Compara com o dia de reunião que isso custa hoje
Custo de API por plano gerado	< US\$ 1,00	Define se o modelo é sustentável
Erro de estimativa em projeto concluído	dentro da faixa de 85% em ≥ 4 de 5 projetos	A prova real. Só medível daqui a meses

A última é a que importa de verdade e é a mais lenta de obter. Comece a registrar o esforço real desde o primeiro projeto planejado com a ferramenta, mesmo que a análise só venha muito depois.

Roadmap

Marcos, critérios de saída e datas projetadas

Base de cálculo: **2 pessoas × 30h/semana = 60h por semana**. Início em 06/08/2026. Os números vêm do [backlog.md](#); se o backlog mudar, recalcule aqui.

Marcos

MARCO	ENTREGA	ESFORÇO	ACUMULADO	DATA PROJETADA
M0	v0 funcionando ponta a ponta	—	—	✅ 05/08/2026
M1	Qualidade do plano com Claude comprovada	37h	37h	11/08/2026
M2	Multiusuário em produção	144h	181h	27/08/2026
M3	Execução do plano	106h	287h	09/09/2026
M4	Ciclo de replanejamento e refinamento	161h	449h	27/09/2026
M5	Dependências e caminho crítico	80h	529h	07/10/2026

Com a faixa de 85% de confiança, M5 cai em **09/10/2026**. É esse o número a levar para fora do time — não o de 07/10.

Mudança de 05/08/2026. A decisão de construir a execução em vez de integrar com Jira ou Linear acrescentou o marco M3 e empurrou os antigos M3 e M4 para M4 e M5. O plano foi de 7,4 para 9,2 semanas no percentil 85. Adição consciente de escopo, registrada para manter a comparação honesta.

M1 — Qualidade do plano comprovada

Critério de saída: cinco projetos reais de domínios diferentes planejados com o Claude, avaliados por uma rubrica escrita, com nota média acima do mínimo que definirmos. Falhas do modelo (recusa, truncamento, tempo esgotado) tratadas na interface com mensagem útil.

Este marco é **bloqueante**. Enquanto ele não fechar, todo o resto está sendo construído em cima de uma premissa não verificada: a de que o Claude produz planos bons o bastante para alguém usar. Se a qualidade não passar, o investimento certo é prompt e avaliação, não login e Gantt.

M2 — Multiusuário em produção

Critério de saída: duas pessoas distintas usam a ferramenta em produção, com dados isolados, rodando em Postgres gerenciado, com migrações versionadas e alerta de erro configurado. Nenhuma consulta retorna dado de outro usuário — e isso está coberto por teste automatizado.

Este é o marco que transforma um projeto local em ferramenta de time.

M3 — Execução do plano

Critério de saída: as 3 pessoas do teste conseguem trabalhar dentro do Kickoff por uma semana inteira sem abrir outra ferramenta. Quadro por estado com arrastar e soltar, responsável por tarefa, comentários, filtros e a visão "o que é meu agora". Tarefa concluída no quadro atualiza o prazo do projeto sozinha.

O teste real deste marco não é a lista de recursos: é **uma semana de uso sem recair no WhatsApp e na planilha**. Se isso não acontecer, o problema é de fluxo, não de recurso faltando — e adicionar recurso vai piorar.

M4 — Ciclo de replanejamento e refinamento

Critério de saída: dá para pedir ao Claude que revise o plano a partir do estado atual (tarefas concluídas, estimativas corrigidas, escopo mudado), ver o que mudou entre as versões e voltar atrás. Edição estrutural completa: adicionar, remover e reordenar épicos e tarefas.

É aqui que a ferramenta deixa de ser "gerador de plano" e vira "companheira de projeto". Um plano só é útil enquanto acompanha a realidade.

M5 — Dependências e caminho crítico

Critério de saída: dependências declaráveis entre tarefas, caminho crítico calculado e um Gantt simples. O prazo passa a considerar sequenciamento, não apenas soma de esforço dividida por capacidade.

Deliberadamente por último. Somar esforço já entrega a maior parte do valor; caminho crítico é refinamento que só compensa depois que as três primeiras frentes estiverem de pé.

O que ficou fora deste roadmap

- Integração ou sincronização com Jira, Linear ou Azure DevOps. **Decidido em 05/08/2026:** construímos a nossa camada de execução (M3).
- Cobrança e planos pagos — entram quando a decisão de vender for tomada.
- App nativo.
- Colaboração em tempo real.
- Sprints, campos personalizados, automações e apontamento de horas. Ver o não-escopo em [produto.md](#): é o que nos impede de virar um Jira pior.

Como este roadmap deve envelhecer

Revisar a cada marco fechado, com três perguntas:

1. As premissas do [README](#) continuam verdadeiras?
2. O esforço real do marco que acabou bateu com a estimativa? Se errou por mais de 30%, ajuste os marcos seguintes na mesma proporção antes de prometer data.

3. Alguma pergunta em aberto foi respondida de um jeito que muda a ordem?

Datas projetadas não são compromisso enquanto o marco anterior não fechou.

Backlog

Épicos e tarefas com estimativa de três pontos

Estimativas em **horas de trabalho efetivo de uma pessoa**, em três pontos: O = otimista, M = mais provável, P = pessimista. A coluna PERT é $(O + 4M + P) / 6$.

Regra que usamos ao estimar: se a pessimista não for pelo menos o dobro da provável em tarefa com incerteza real, a pessimista está otimista.

E1 — Validar o caminho do Claude ● bloqueante

Sem isto, tudo o mais é construído sobre premissa não verificada.

#	TAREFA	PAPEL	PRIO	O	M	P	PERT
1.1	Configurar chave e gerar 5 planos reais de domínios diferentes	produto	alta	2	4	8	4,3
1.2	Escrever a rubrica de avaliação de plano e aplicá-la aos 5	produto	alta	6	12	24	13,0
1.3	Ajustar o prompt do sistema com base nas falhas observadas	backend	alta	4	10	24	11,3
1.4	Tratar falhas do modelo na interface (recusa, truncamento, timeout)	frontend	alta	4	8	16	8,7

Total do épico: 37h

A rubrica (1.2) é o entregável mais valioso desta frente. Sem ela, "o plano ficou bom" é opinião, e não dá para saber se uma mudança de prompt melhorou ou piorou. Critérios sugeridos: cobre as frentes esquecidas por padrão? as tarefas cabem em menos de uma semana? as pessimistas são realmente pessimistas? os riscos são específicos do domínio ou genéricos?

E2 — Contas e multiusuário

#	TAREFA	PAPEL	PRIO	O	M	P	PERT
2.1	Escolher e integrar provedor de autenticação	backend	alta	6	12	24	13,0
2.2	Adicionar <code>owner_id</code> ao schema e migrar os dados existentes	dados	alta	4	8	16	8,7
2.3	Filtrar todas as consultas por usuário, com teste que prova o isolamento	backend	alta	4	8	20	9,3
2.4	Telas de login, sessão e saída	frontend	alta	6	12	20	12,3
2.5	Espaço de trabalho compartilhado: convidar pessoas para um projeto	backend	media	12	24	48	26,0

Total do épico: 69h

2.3 é a tarefa que mais assusta e a mais fácil de subestimar. Vazamento de dado entre usuários é falha silenciosa: nada quebra, só aparece o dado errado. O teste automatizado não é opcional aqui.

E3 — Replanejamento

#	TAREFA	PAPEL	PRIO	O	M	P	PERT
3.1	Ação "revisar plano" que envia o estado atual ao Claude	backend	alta	12	24	48	26,0
3.2	Comparação visual entre a versão anterior e a revisada	frontend	alta	12	24	40	24,7
3.3	Histórico de versões do plano, com volta atrás	dados	media	8	16	32	17,3

Total do épico: 68h

3.1 tem a maior incerteza do backlog. O problema difícil não é chamar a API de novo: é decidir o que preservar. Se a pessoa corrigiu 20 estimativas à mão e a revisão as descarta, ela nunca mais usa o botão.

E4 — Dependências e caminho crítico

#	TAREFA	PAPEL	PRIO	O	M	P	PERT
4.1	Modelar dependências entre tarefas no schema	dados	media	6	12	24	13,0
4.2	Interface para declarar e visualizar dependências	frontend	media	8	16	28	16,7
4.3	Cálculo de caminho crítico e folga por tarefa	backend	media	12	24	48	26,0
4.4	Gráfico de Gantt simples	frontend	baixa	12	24	40	24,7

Total do épico: 80h

4.1 precisa impedir ciclo de dependência na escrita, não só na leitura. Detectar ciclo depois que ele existe no banco é problema muito pior.

E5 — Confiabilidade e operação

#	TAREFA	PAPEL	PRIO	O	M	P	PERT
5.1	Geração assíncrona com estado de progresso na tela	backend	alta	12	24	40	24,7
5.2	Testes automatizados do caminho crítico	qa	alta	12	24	40	24,7
5.3	Migrações versionadas e implantação em Postgres gerenciado	devops	alta	6	12	24	13,0
5.4	Observabilidade e custo de API por plano gerado	devops	media	6	12	20	12,3

Total do épico: 75h

5.1 deixa de ser conforto e vira necessidade em produção: hoje a geração acontece dentro da requisição, e plataformas de hospedagem cortam requisição longa. Com o Claude em esforço alto, dois minutos é normal.

E6 — Refinamento do produto

#	TAREFA	PAPEL	PRIO	O	M	P	PERT
6.1	Edição estrutural: adicionar, remover e reordenar épicos e tarefas	frontend	alta	12	24	40	24,7
6.2	Exportação para CSV e para uma ferramenta de gestão (a definir)	backend	media	16	32	64	34,7
6.3	Modo apresentação para a reunião de abertura do projeto	frontend	baixa	8	16	28	16,7
6.4	Acessibilidade e ajuste fino em telas pequenas	frontend	media	8	16	28	16,7

Total do épico: 93h

6.2 está com pessimista de 64h de propósito: enquanto não decidirmos o destino (Jira? Linear? Azure DevOps?), a incerteza é real. Assim que houver decisão, reestime — provavelmente para baixo.

E7 — Execução do plano

Entrou em 05/08/2026, com a decisão de não integrar com Jira ou Linear e construir a camada de execução aqui dentro. É o épico que transforma o plano em lugar de trabalho.

#	TAREFA	PAPEL	PRIO	O	M	P	PERT
7.1	Quadro por estado com arrastar e soltar	frontend	alta	16	32	64	34,7
7.2	Responsável por tarefa	backend	alta	6	12	24	13,0
7.3	Comentários por tarefa	backend	media	12	24	40	24,7
7.4	Filtros e visão "minhas tarefas"	frontend	alta	8	16	28	16,7
7.5	Aviso de mudança relevante, dentro do app	backend	media	8	16	32	17,3

Total do épico: 106h

7.1 é a tarefa de maior incerteza do backlog inteiro, com pessimista de 64h. Arrastar e soltar acessível — que funcione com teclado e leitor de tela — é muito mais trabalho do que a versão que só funciona com mouse. Decida qual das duas você quer **antes** de começar, não no meio.

O histórico de alterações não está aqui de propósito: ele já existe como tarefa 3.3, no épico de replanejamento. Não duplicar.

Totais

	VALOR
Esforço esperado (soma PERT)	529h
Desvio-padrão agregado	23h
Esforço com 85% de confiança	553h
Duração esperada (60h/semana)	8,8 semanas
Duração com 85% de confiança	9,2 semanas

O épico E7 acrescentou **106h** ao plano — de 422h para 529h, e de 7,4 para 9,2 semanas no percentil 85. Foi uma adição consciente de escopo, não um estouro: está registrada aqui para que a comparação com a estimativa original continue possível.

Uma ressalva honesta sobre esses números

A agregação PERT assume que as tarefas são **independentes** entre si. Não são. Se a integração com o provedor de autenticação for pior que o esperado, é provável que o teste de isolamento também seja — a mesma causa afeta as duas. Na prática isso significa que o desvio-padrão de 20h está **subestimado**, e a faixa de 85% é um piso, não um teto.

Como usar isso sem se enganar:

- Trate 7,4 semanas como o cenário bom, não como o pior caso.
- O único corretivo que funciona é medir: registre o esforço real de cada tarefa concluída e compare com a estimativa. Depois de uns 20 pontos de dados, o histórico do time vale mais que qualquer fórmula.

Riscos

O que pode dar errado e o que fazemos a respeito

Ordenados por impacto × probabilidade, ambos de 1 a 5. Revisar a cada marco fechado: risco que não se revisita vira decoração.

● 20 — A qualidade do plano gerado não é boa o suficiente para alguém usar

Impacto 5 · Probabilidade 4

É o risco existencial do projeto. Nunca rodamos o caminho do Claude de verdade — todos os planos até hoje vieram da heurística local. Se a saída do modelo for genérica ("implementar backend", "fazer testes"), a ferramenta não tem razão de existir, e nenhuma quantidade de interface bonita salva.

Mitigação: E1 é o primeiro épico e é bloqueante. A rubrica de avaliação (1.2) vem antes de qualquer investimento em login ou Gantt. Se a nota não passar do mínimo, o roadmap para e vira trabalho de prompt e avaliação.

Sinal de alerta: planos gerados que a pessoa apaga ou reescreve inteiro.

● 16 — Vazamento de dado entre usuários quando o multiusuário entrar

Impacto 5 · Probabilidade 3 (com o teste em 2.3: cai para 1)

Toda consulta hoje é global. Adicionar `owner_id` não protege nada sozinho — proteção é filtrar em **todas** as consultas, e é fácil esquecer uma. A falha é silenciosa: nada quebra, só aparece o projeto de outra pessoa.

Mitigação: a tarefa 2.3 inclui teste automatizado que cria dois usuários, cria dado com cada um e prova que nenhuma consulta atravessa a fronteira. Esse teste roda na integração contínua e é bloqueante para publicar.

Sinal de alerta: qualquer consulta nova em `src/db/queries.ts` sem cláusula de usuário.

● 16 — Virarmos um Jira pior em vez de uma ferramenta mais simples

Impacto 4 · Probabilidade 4

Risco novo, criado pela decisão de 05/08/2026 de construir a execução em vez de integrar. O caminho para o fracasso é conhecido e confortável: cada recurso que falta parece razoável isoladamente. Falta sprint. Falta campo personalizado. Falta automação. Dois meses depois temos um Jira com um décimo dos recursos e nenhuma vantagem — porque a vantagem prometida era **ser mais simples**.

Mitigação: o não-escopo em [produto.md](#) lista explicitamente o que não construímos, e cada item ali é uma linha que só se cruza com discussão aberta. O critério de saída do M3 não é lista de recursos: é as 3 pessoas trabalharem uma semana inteira sem abrir outra ferramenta.

Sinal de alerta: o primeiro pedido de "só um campinho a mais" aceito sem discussão.

● 10 — Custo de API por plano inviabiliza o produto pago

Impacto 5 · Probabilidade 2

Rodamos Claude Opus 5 com esforço alto e `max_tokens` de 32000, e nunca medimos o custo real de uma geração. Foi decidido em 05/08/2026 que **não há teto de orçamento por enquanto** — o que reduz a probabilidade agora, com 3 pessoas em teste, mas não elimina o problema: se a decisão de vender vier e o custo por plano for alto demais, o preço não fecha.

Mitigação: a tarefa 5.4 mede custo por plano gerado mesmo sem teto definido. Medir é barato agora; descobrir na negociação de preço é caro. Alavancas se o número assustar: baixar o esforço para `medium` e comparar qualidade pela rubrica; cache de prompt no prompt de sistema, que é fixo; limite de gerações.

Sinal de alerta: a conversa sobre virar produto começar sem esse número na mão.

● 12 — Geração dentro da requisição estoura o tempo limite em produção

Impacto 4 · Probabilidade 3

Hoje a geração acontece na própria ação do servidor. Localmente não incomoda; em plataforma de hospedagem, requisição longa é cortada. Com esforço alto, dois minutos é comum.

Mitigação: tarefa 5.1 move a geração para trabalho assíncrono com estado de progresso, antes do marco M2, que é quando isso vai para produção.

Sinal de alerta: primeira publicação em ambiente hospedado.

● 12 — O botão de replanejar descarta o trabalho manual da pessoa

Impacto 4 · Probabilidade 3

Se a pessoa corrigiu vinte estimativas à mão e a revisão do plano as substitui, ela usa o botão uma vez e nunca mais. O recurso mais caro do E3 vira recurso morto.

Mitigação: decidir a política de preservação **antes** de codar a 3.1, e escrevê-la no [decisoes-tecnicas.md](#). Ponto de partida sugerido: tarefa tocada pela pessoa é imutável; o modelo só pode adicionar, remover o que não foi tocado e sugerir — nunca sobrescrever edição humana. A comparação visual (3.2) existe para tornar isso auditável.

● 9 — Time sem a disponibilidade real que o plano assume

Impacto 3 · Probabilidade 3

Todo o roadmap assume 2 pessoas × 30h efetivas. Trinta horas efetivas é uma semana inteira sem reunião longa, sem suporte e sem troca de contexto. É uma premissa agressiva.

Mitigação: confirmar a disponibilidade real antes de comunicar qualquer data para fora. Se for 20h por pessoa, o M4 vai de 24/09 para meados de outubro — e é melhor descobrir isso agora do que em setembro.

● 9 — Escopo crescendo por pedidos fora do combinado

Impacto 3 · Probabilidade 3

"Já que estamos nisso, dá para colocar um kanban?" O não-escopo da v1 existe justamente porque cada um desses pedidos é razoável isoladamente.

Mitigação: a seção de não-escopo em [produto.md](#) é para ser citada. Todo pedido novo é tratado como **troca**, não como adição: entra quando algo de peso equivalente sai, e a data se move de forma explícita.

● 8 — Dependência de um único fornecedor de modelo

Impacto 4 · Probabilidade 2

Todo o valor do produto passa pela API da Anthropic. Indisponibilidade, mudança de preço ou de política afeta a ferramenta inteira.

Mitigação: o motor heurístico local já é uma degradação graciosa — o produto não para, só perde qualidade. A camada de geração já é isolada em `src/lib/planner.ts` com saída validada por schema, o que torna a troca de modelo um trabalho contido. Não vale investir mais que isso agora.

Decisões técnicas

Arquitetura já definida, com motivo e gatilho de revisão

Registro das decisões de arquitetura já tomadas, com o motivo e o que justificaria voltar atrás. Decisão sem motivo escrito vira dogma; decisão sem gatilho de revisão vira dívida.

DT-01 — Next.js com App Router e Server Actions

Status: aceita · 05/08/2026

Renderização no servidor com mutações via Server Actions, sem camada de API própria.

Por quê: o produto é essencialmente CRUD sobre um plano, com uma chamada pesada de IA no servidor. Server Actions eliminam a escrita de endpoints e a duplicação de validação entre cliente e servidor. O mesmo schema Zod valida o formulário e o corpo da ação.

Custo aceito: acoplamento ao Next. Se um dia houver app mobile nativo, será preciso expor uma API de verdade.

Revisar se: aparecer um segundo consumidor dos dados que não seja o navegador.

DT-02 — Drizzle ORM com PGLite em desenvolvimento e Postgres em produção

Status: aceita · 05/08/2026

`DATABASE_URL` ausente → PGLite (Postgres compilado em WASM, arquivo local em `.pglite/`). Presente → Postgres via `postgres-js`. O mesmo schema Drizzle serve aos dois, porque o dialeto é o mesmo.

Por quê: `npm install && npm run dev` e o projeto sobe. Sem Docker, sem serviço para instalar, sem SQLite fingindo ser Postgres com diferenças sutis de tipo que aparecem só em produção.

Custo aceito: PGLite é de processo único; não serve para produção.

Revisar se: precisarmos de extensão Postgres que o PGLite não tenha (pesquisa vetorial, por exemplo).

DT-03 — Tabelas criadas por bootstrap em vez de migrações versionadas

Status: aceita, com prazo de validade · 05/08/2026

`src/db/index.ts` roda `CREATE TABLE IF NOT EXISTS` na primeira conexão.

Por quê: para a v0, o passo de migração é atrito sem benefício — só existe um ambiente e ele é descartável.

Custo aceito: não há caminho de evolução do schema. Adicionar coluna a uma tabela existente não acontece.

Revisar quando: tarefa 5.3, obrigatoriamente antes do M2. `drizzle.config.ts` já está pronto para `drizzle-kit generate / push`. **Esta decisão expira no primeiro dado real que não puder ser perdido.**

DT-04 — Saída estruturada com schema Zod em vez de texto interpretado

Status: aceita · 05/08/2026

O plano vem do Claude por `output_config.format` com o schema de `src/lib/plan-schema.ts`, e é validado por Zod antes de tocar o banco.

Por quê: interpretar Markdown gerado por modelo é frágil e falha de formas difíceis de detectar. Com schema, ou o plano chega no formato certo, ou não chega — e o erro é explícito. O mesmo schema documenta o contrato para quem lê o código e descreve os campos para o modelo, via `.describe()`.

Custo aceito: o schema é rígido. Adicionar um campo exige mudar schema, banco e interface juntos.

DT-05 — Motor heurístico local como degradação graciosa

Status: aceita · 05/08/2026

Sem `ANTHROPIC_API_KEY`, ou quando a chamada falha, `src/lib/heuristic-planner.ts` monta um plano a partir de blocos ativados por palavras da descrição.

Por quê: três motivos. O produto continua funcionando quando a API está fora; dá para desenvolver e testar toda a interface sem gastar com API; e um novo integrante clona o repositório e vê o produto funcionando sem pedir credencial a ninguém.

Custo aceito, e é um custo real: cria a ilusão de que o sistema funciona quando o caminho principal nunca foi exercitado. Foi exatamente o que aconteceu na v0. Por isso a origem do plano é gravada na coluna `source` e mostrada na interface — **um plano com `source = 'heurístico'` não é prova de que o Claude funciona.**

DT-06 — Estimativa de três pontos com agregação PERT

Status: aceita · 05/08/2026

Cada tarefa carrega otimista, provável e pessimista. Esperado por tarefa é $(O + 4M + P) / 6$; o agregado soma esperados, soma **variâncias** (não desvios) e apresenta a faixa de 85% como esperado + $1,04\sigma$.

Por quê: é o único jeito honesto de comunicar prazo. Um número único vira compromisso sem ninguém dizer qual era a confiança por trás dele.

Limitação conhecida e assumida: PERT assume independência entre tarefas. Tarefas de projeto real são correlacionadas, então o desvio-padrão agregado é subestimado e a faixa de 85% é um piso. Está escrito em [backlog.md](#) e deve ser dito em voz alta quando o número for apresentado.

Revisar se: tivermos histórico suficiente de esforço real para calibrar com dados do time em vez de fórmula.

DT-07 — Interface só em tema escuro

Status: aceita · 05/08/2026

Por quê: compromisso deliberado de design. Um tema bem feito é melhor que dois medianos, e é metade do trabalho de estilo e de teste.

Revisar se: houver pedido real de pessoa usuária, ou se surgir requisito de acessibilidade que o tema escuro não atenda.

DT-08 — Política de preservação no replanejamento

Status: ● pendente — decidir antes de codar a tarefa 3.1

Quando o Claude revisa um plano existente, o que ele pode sobrescrever?

Proposta a validar: toda tarefa tocada por uma pessoa (estimativa editada, estado alterado, título mudado) é imutável para o modelo. Ele pode adicionar tarefas novas, remover tarefas que ninguém tocou e sugerir mudanças em tarefas tocadas — nunca aplicá-las sozinho. A comparação visual da tarefa 3.2 existe para tornar isso auditável antes de confirmar.

Fechar esta decisão é pré-requisito do épico E3. Ver o risco de mesmo tema em [riscos.md](#).

Processo

Cadência, definição de pronto, testes e escopo

Time de 2 pessoas. O processo é proporcional a isso: o suficiente para não perder trabalho e não repetir discussão, nada além.

Cadência

RITUAL	QUANDO	DURAÇÃO	PARA QUÊ
Alinhamento diário	Toda manhã	10 min	O que travou. Não é relatório de status — isso está no quadro
Revisão de marco	Ao fechar cada M	45 min	Esforço real × estimado, riscos, ajuste do roadmap
Repriorização	Início de cada marco	30 min	Confirmar que a ordem do backlog ainda faz sentido

Sem sprint de duas semanas. Com dois marcos deste tamanho, cerimônia de sprint custa mais do que entrega.

Fluxo de trabalho

1. Pegue a tarefa mais alta do épico em andamento. **Não pule épico** — a ordem em `backlog.md` reflete dependência real, não preferência.
2. Branch a partir de `main`: `tipo/descricao-curta` (`feat/auth-provider` , `fix/pglite-bootstrap` , `docs/rubrica-avaliacao`).
3. Commits pequenos, no imperativo, em português: `adiciona owner_id ao schema de projetos` .
4. Abra o PR quando estiver pronto para revisão, não antes. PR aberto é convite a revisar.
5. **Revisão obrigatória**, mesmo com duas pessoas. Especialmente com duas pessoas — é a única barreira que sobra.
6. `main` sempre publicável.

Definição de pronto

Uma tarefa só é marcada como concluída quando **todos** os itens valem:

- ☐ Faz o que a descrição da tarefa diz, incluindo o critério de aceite.
- ☐ `npm run build` passa.
- ☐ `npx tsc --noEmit` passa sem erro.
- ☐ `npm run lint` passa sem aviso novo.

- ☐ Testado no navegador, no fluxo real — não só "compila".
- ☐ Se mexeu em consulta ao banco depois do M2: tem teste provando o isolamento por usuário.
- ☐ Se mudou decisão de arquitetura: registrada em [decisoes-tecnicas.md](#).
- ☐ Se a estimativa errou por mais de 50%: o esforço real está anotado na tarefa. **Este item é o que vai calibrar as estimativas futuras — não pule.**

Testes

Não perseguimos cobertura. Persequimos as coisas que, quebrando, param o produto:

O QUE	COMO	A PARTIR DE
Cálculo PERT e agregação	Teste unitário de <code>src/lib/estimate.ts</code>	agora
Validação da saída do modelo	Teste de <code>PlanSchema</code> com respostas reais e malformadas	E1
Isolamento entre usuários	Teste de integração com dois usuários	E2, bloqueante
Fluxo descrever → gerar → editar	Um teste ponta a ponta, só o caminho feliz	E5

Nada disso existe hoje. A tarefa 5.2 é onde entra, e ela vem antes do M2.

Como registrar esforço real

É a prática que sustenta todas as estimativas futuras, e a mais fácil de abandonar. Ao fechar cada tarefa, anote no PR uma linha:

```
Estimado (PERT): 13h · Real: 21h · Motivo: a documentação do provedor
estava desatualizada e o fluxo de callback mudou.
```

Depois de uns vinte pontos, o histórico do time vale mais que qualquer fórmula. O motivo importa mais que o número: é ele que revela o padrão.

Quando a estimativa fura

Furar estimativa não é problema — não avisar é.

- **Até 30% acima:** siga, anote o real ao fechar.
- **Entre 30% e 100%:** avise a outra pessoa no alinhamento do dia. Reavalie as tarefas parecidas do mesmo épico.
- **Acima de 100%:** pare. Não é mais uma tarefa mal estimada, é uma tarefa mal entendida. Reabra o escopo dela antes de continuar codando.

Quando pedirem algo fora do escopo

Cite o não-escopo em [produto.md](#) e trate como **troca**, não como adição:

"Cabe sim. Entra no lugar de quê? Se nada sair, a data do M3 anda duas semanas — sua escolha."

Isso não é burocracia. É a única forma de o número que você comunicou continuar significando alguma coisa daqui a um mês.